Creating a VPC using Terraform and Configuring EC2 using Ansible

Aim:

To create an AWS Virtual Private Cloud (VPC) using Terraform and configure EC2 instances automatically using Ansible

Scenario

An organization wants to deploy applications on AWS with minimal manual work. They require:

- A custom VPC
- Public subnet with internet access
- EC2 instances
- Automated server configuration

Terraform is used for infrastructure creation, and Ansible is used for configuration management

Tools & Technologies

- AWS (EC2, VPC)
- Terraform
- Ansible
- Linux
- SSH

Architecture Description

- One VPC
- One public subnet
- One Internet Gateway
- One route table
- Two EC2 instances
- Ansible configures EC2 instances after creation

Procedure

Step 1: Install Terraform Verify Terraform installation:

bash

terraform --version

Step 2: Create Terraform Project Structure

text

terraform-vpc/

|— main.tf

|— variables.tf

|— outputs.tf

Step 3: Configure AWS Provider

hcl

```
provider "aws" {
  region = "us-east-1"
}
```

Step 4: Create VPC

hcl

```
resource "aws_vpc" "dev_vpc" {
```

```
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "Dev-VPC"
  }
}
```

Step 5: Create Public Subnet

```
hcl
resource "aws_subnet" "public_subnet" {
  vpc_id     = aws_vpc.dev_vpc.id
  cidr_block = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone = "us-east-1a"
}
```

Step 6: Create Internet Gateway

```
hcl
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.dev_vpc.id
}
```

Step 7: Create Route Table

```
hcl
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.dev_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
```

Step 8: Associate Route Table

```
hcl
resource "aws_route_table_association" "rt_assoc" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}
```

Step 9: Create Security Group

```
hcl
resource "aws_security_group" "sg" {
  vpc_id = aws_vpc.dev_vpc.id

  ingress {
    from_port   = 22
    to_port     = 22
```

```
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }
}
```

Step 10: Create EC2 Instances

```
hcl
resource "aws_instance" "ec2" {
 count        = 2
 ami          = "ami-0c02fb55956c7d316"
 instance_type = "t2.micro"
 subnet_id     = aws_subnet.public_subnet.id
 key_name      = "my-key"
 security_groups = [aws_security_group.sg.id]
}
```

Step 11: Execute Terraform Commands

```
bash
terraform init
terraform plan
terraform apply
```

Output: VPC and EC2 instances are created successfully.

Configuration Using Ansible

Step 12: Create Ansible Inventory

```
ini
[servers]
server1 ansible_host=<EC2_PUBLIC_IP_1>
server2 ansible_host=<EC2_PUBLIC_IP_2>

[servers:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=my-key.pem
```

Step 13: Test Connectivity

```
bash
ansible servers -m ping
```

Step 14: Create Ansible Playbook

```
yaml
- name: Configure EC2 Instances
```

```
hosts: servers
become: yes

tasks:
 - name: Install Docker
   yum:
     name: docker
     state: present

 - name: Start Docker Service
   service:
     name: docker
     state: started
     enabled: true
```

Step 15: Run Ansible Playbook

```
bash
ansible-playbook install_docker.yml
```

Result

- AWS VPC and EC2 instances were created using Terraform
- EC2 instances were configured automatically using Ansible
- Docker service was installed and started successfully

Conclusion

This experiment demonstrates how Terraform automates infrastructure creation and Ansible automates server configuration, following real-world DevOps practices.